Шпаргалка Neovim: клавиши и команды

Конфигурация для Windows 11: Python, Rust, PowerShell, Markdown, LaTeX, локальный ИИ-помощник · октябрь 2026

Как читать

- **`Esc`** включает обычный режим (команды), **`i`** включает режим вставки (печать). Если не сказано иное, клавиши ниже нажимаются в обычном режиме.
- **`Space a`** означает: нажать пробел, отпустить, нажать **`a`**.
- Всё, что начинается с двоеточия, это команда: набрать её и нажать Enter.
- Слова вроде **`word`**, **`old`**, **`new`**, **`file`** в командах это подстановки: замените их своими.
- Пути в PowerShell можно писать с прямыми слэшами: **`$env:LOCALAPPDATA/nvim`** работает так же, как с обратными.

Neovim cheat sheet: keys and commands

Configuration for Windows 11: Python, Rust, PowerShell, Markdown, LaTeX, a local AI assistant · October 2026

How to read

- **`Esc`** switches to normal mode (commands), **`i`** switches to insert mode (typing). Unless stated otherwise, the keys below are pressed in normal mode.
- **`Space a`** means: press the space bar, release it, press **`a`**.
- Anything that starts with a colon is a command: type it and press Enter.
- Words such as **`word`**, **`old`**, **`new`**, **`file`** in commands are placeholders: replace them with your own.
- In PowerShell you can write paths with forward slashes: **`$env:LOCALAPPDATA/nvim`** works the same as with backslashes.

Режимы / Modes

Keys / Клавиши	Что делает	What it does
`i`	Режим вставки перед курсором	Insert mode before the cursor
`o / O`	Новая строка ниже / выше и режим вставки	New line below / above and insert mode
`Esc`	Выход в обычный режим	Back to normal mode
`jk`	В режиме вставки, нажатые быстро: то же, что Esc	In insert mode, pressed quickly: same as Esc
`V`	Выделить строки целиком (затем j / k растягивают выделение)	Select whole lines (then j / k extend the selection)
`v`	Выделить по символам	Select by characters

Сохранение и выход / Saving and quitting

Command / Команда	Что делает	What it does
`:w`	Сохранить	Save
`:wq`	Сохранить и выйти	Save and quit
`:q`	Выйти (не сработает, если есть несохранённые изменения)	Quit (will not work if there are unsaved changes)
`:q!`	Выйти без сохранения	Quit without saving
`:qa`	Закрыть все окна	Close all windows
`:qa!`	Закрыть все окна без сохранения (выручает, когда открыты чат или дерево)	Close all windows without saving (helps when the chat or the tree is open)
`:e!`	Выбросить несохранённые изменения и вернуть файл как при последнем сохранении	Discard unsaved changes and restore the file as of the last save

Перемещение / Moving around

Keys / Клавиши	Что делает	What it does
`j / k`	Вниз / вверх	Down / up
`0`	В начало строки	To the start of the line
`gg`	В начало файла	To the start of the file
`G`	В конец файла	To the end of the file

Keys / Клавиши	Что делает	What it does
`:30 or 30G`	На строку 30	To line 30
`Ctrl+Home`	В начало файла	To the start of the file
`Ctrl+w, w`	Переключиться между окнами (файл, дерево, чат)	Switch between windows (file, tree, chat)

Поиск и правка текста / Searching and editing text

Keys / Клавиши	Что делает	What it does
`/word, Enter`	Найти слово в файле	Find a word in the file
`:44d`	Удалить строку 44	Delete line 44
`:25,$d`	Удалить строки с 25 до конца файла	Delete lines from 25 to the end of the file
`:%s/old/new/`	Заменить текст в файле (первое совпадение в каждой строке)	Replace text in the file (first match on each line)
`cw`	Заменить слово под курсором: удаляет слово и включает вставку	Replace the word under the cursor: deletes the word and starts insert mode
`0, D`	Очистить строку целиком (D удаляет от курсора до конца строки)	Clear the whole line (D deletes from the cursor to the end of the line)
`gg=G`	Выровнять отступы во всём файле	Re-indent the whole file
`u`	Отменить последнее действие (нажимать можно несколько раз)	Undo the last action (can be pressed several times)
`Ctrl+r`	Вернуть отменённое	Redo what was undone

Файлы и поиск / Files and search

Keys / Клавиши	Что делает	What it does
`:e $MYVIMRC`	Открыть init.lua	Open init.lua
`:e $LOCALAPPDATA/nvim/lua/config/lazy.lua`	Открыть файл по пути	Open a file by path
`Ctrl+p`	Поиск файла по имени (вместо стандартной клавиши)	Find a file by name (replaces the default key)
`Ctrl+f`	Поиск текста по всем файлам проекта (вместо прокрутки страницы)	Search text across all project files (replaces page scrolling)
`Ctrl+n`	Открыть и закрыть файловое дерево слева	Open and close the file tree on the left
`Arrows, Enter, Esc`	В окнах поиска и в дереве: выбрать, открыть, закрыть	In the search windows and the tree: choose, open, close

Файлы конфигурации лежат в `$LOCALAPPDATA/nvim/`: `init.lua`, `lua/config/options.lua`, `lua/config/lazy.lua` (плагины), `lua/config/lsp.lua` (языковые серверы), `lazy-lock.json` (версии плагинов).

The configuration files live in `$LOCALAPPDATA/nvim/`: `init.lua`, `lua/config/options.lua`, `lua/config/lazy.lua` (plugins), `lua/config/lsp.lua` (language servers), `lazy-lock.json` (plugin versions).

Код: подсказки и автодополнение / Code: hints and completion

Keys / Клавиши	Что делает	What it does
`gd`	Перейти к определению функции или переменной (Python, Rust, PowerShell)	Go to the definition of a function or variable (Python, Rust, PowerShell)
`Ctrl+y`	Принять выбранный вариант	Accept the selected item
`Ctrl+n / Ctrl+p / arrows`	Выбрать вариант в меню	Choose an item in the menu
`Ctrl+e`	Закрыть меню	Close the menu
`Ctrl+Space`	Вызвать меню вручную	Open the menu manually

Ошибки и предупреждения от языкового сервера показываются текстом справа от строки. Запись `E:2 W:1` в строке статуса показывает количество ошибок и предупреждений в файле. Меню автодополнения появляется само при наборе.

Errors and warnings from the language server are shown as text to the right of the line. The entry `E:2 W:1` in the status line shows the number of errors and warnings in the file. The completion menu appears by itself while you type.

Note: in the default blink.cmp key layout the Enter key does

Примечание: в стандартной раскладке плагина blink.cmp клавиша Enter вариант не принимает, для этого нужен **Ctrl+y**.

Сервер запускается через 10–30 секунд после открытия файла. Если подсказок нет, выполните **:checkhealth vim.lsp**.

not accept an item; use **Ctrl+y** for that.

The server starts 10–30 seconds after you open a file. If there are no hints, run **:checkhealth vim.lsp**.

Markdown

Command / Команда	Что делает	What it does
:RenderMarkdown toggle	Включить или выключить оформленный вид Markdown	Turn the rendered Markdown view on or off
:set ft?	Какой тип файла определил Neovim (для Markdown должно быть filetype=markdown)	Which file type Neovim detected (for Markdown it should be filetype=markdown)

Оформление показывается в обычном режиме, а в режиме вставки виден исходный текст разметки. Языковой сервер для Markdown называется marksman; подключён ли он, показывает **:checkhealth vim.lsp**.

The rendered view is shown in normal mode, while insert mode shows the raw markup. The language server for Markdown is called marksman; whether it is attached is shown by **:checkhealth vim.lsp**.

LaTeX (VimTeX)

Keys / Клавиши	Что делает	What it does
\ll	Запустить непрерывную сборку PDF	Start continuous PDF build
\lv	Открыть PDF	Open the PDF
\le	Показать ошибки сборки	Show build errors
\lk	Остановить сборку	Stop the build
\li	Информация VimTeX (проект, компилятор)	VimTeX info (project, compiler)
\lc	Удалить служебные файлы сборки (.aux, .log и др.)	Remove build auxiliary files (.aux, .log, etc.)
:VimtexInfo	То же, что **\li**, командой	Same as **\li**, as a command
:Mason	Окно менеджера языковых серверов	Language server manager window
:MasonInstall texlab marksman	Установить серверы для LaTeX и Markdown	Install the LaTeX and Markdown servers

Знак **** это клавиша Backslash. Для сборки нужны MiKTeX и Perl; подробности в инструкции **install-markdown-latex**.

The sign **** is the Backslash key. Building requires MiKTeX and Perl; see the guide **install-markdown-latex** for details.

ИИ-помощник (локальная модель)

Перед работой Ollama должна быть запущена (значок ламы в трее). Модель: **qwen2.5-coder:7b**, контекст 16384 токена.

AI assistant (local model)

Ollama must be running before you start (the llama icon in the tray). Model: **qwen2.5-coder:7b**, context 16384 tokens.

Keys / Клавиши	Что делает	What it does
Space a	Открыть и закрыть чат	Open and close the chat
V, j/k, Space c	Отправить выделенный код в чат	Send the selected code to the chat
V, j/k, Space i	Правка выделенного кода по вашей задаче	Edit the selected code according to your task
i, Ctrl+s	В чате: начать печатать и отправить вопрос	In the chat: start typing and send the question
g2 / g3 / g1	Принять / отклонить / всегда принимать правку	Accept / reject / always accept the edit
ga	В чате: выбрать другую модель	In the chat: choose another model

Команды: **:CodeCompanionChat** (открыть чат), **:CodeCompanionChat Toggle**, **:CodeCompanionChat Add** (добавить выделение), **:CodeCompanion task text** (правка на месте; вместо **task text** напишите задачу).

Клавиши **g1**, **g2**, **g3** и **ga** назначены самим плагином codecompanion.nvim; в других версиях плагина они могут быть другими.

Commands: **:CodeCompanionChat** (open the chat), **:CodeCompanionChat Toggle**, **:CodeCompanionChat Add** (add the selection), **:CodeCompanion task text** (edit in place; write your task instead of **task text**).

The keys **g1**, **g2**, **g3** and **ga** are assigned by the codecompanion.nvim plugin itself; other versions of the plugin may use different ones.

Вопрос по коду. Выделить код (`V`, `j`/`k`), `Space c`, `i`, вопрос, `Ctrl+s`. Ответ приходит через 10–20 секунд.

Правка кода. Выделить код, `Space i`, написать задачу, Enter. Правка появится отдельным блоком: `g2` принять, `g3` отклонить. Результат всегда стоит просмотреть: 7B-модель иногда ошибается в деталях (например, ставит комментарий не туда).

Совет: выделяйте нужную функцию, а не весь файл. Так ответы точнее, а контекст не переполняется. Проверка работы на видеокарте: `ollama ps` (должно быть `100% GPU`).

A question about code. Select the code (`V`, `j`/`k`), `Space c`, `i`, the question, `Ctrl+s`. The answer arrives in 10–20 seconds.

Editing code. Select the code, `Space i`, write the task, Enter. The edit appears as a separate block: `g2` accepts, `g3` rejects. Always review the result: a 7B model sometimes gets details wrong (for example, puts a comment in the wrong place).

Tip: select the function you need rather than the whole file. Answers are more accurate and the context does not overflow. To check that it runs on the video card: `ollama ps` (it should say `100% GPU`).

Диагностика и служебные команды / Diagnostics and utility commands

Command / Команда	Что делает	What it does
`:checkhealth`	Общая проверка состояния	General health check
`:checkhealth vim.lsp`	Работают ли языковые серверы	Are the language servers working
`:checkhealth lazy`	Состояние менеджера плагинов	Plugin manager state
`:checkhealth nvim-treesitter`	Установленные парсеры подсветки	Installed highlighting parsers
`:checkhealth vim.provider`	Проверка провайдеров (Python, Node и др.)	Provider check (Python, Node, etc.)
`:Inspect`	Чем подсвечено слово под курсором (подтверждает работу Treesitter)	What highlights the word under the cursor (confirms Treesitter works)
`:set ft?`	Какой тип файла определил Neovim	Which file type Neovim detected
`:TSInstall python`	Установить парсер подсветки для языка (также rust, lua)	Install a highlighting parser for a language (also rust, lua)
`:Lazy`	Окно менеджера плагинов; q закрывает его	Plugin manager window; q closes it

Команды PowerShell, которые использовались / PowerShell commands that were used

Command / Команда	Зачем	What for
`nvim file`	Открыть файл в Neovim	Open a file in Neovim
`nvim $env:LOCALAPPDATA/nvim/lua/config/lsp.lua`	Открыть файл конфигурации	Open a configuration file

`pwsh --version`
`rustc --version`
`cargo --version`
	Проверить установленные программы	Check installed programs

`rust-analyzer --version` `pyright --version`	Проверить языковые серверы	Check the language servers
`cargo new rust_test`	Создать тестовый проект на Rust	Create a test Rust project
`nvidia-smi`	Объём видеопамяти и загрузка видеокарты	Video memory and GPU load
`ollama pull qwen2.5-coder:7b`	Скачать модель	Download the model
`ollama run qwen2.5-coder:7b`	Задать вопрос модели в терминале	Ask the model a question in the terminal
`ollama ps`	Какие модели загружены и на чём работают	Which models are loaded and what they run on

`git init`
`git add .`
`git commit -m 'message'`
	Создать репозиторий и сохранить версию конфига	Create a repository and save a config version

`git log --oneline` `git status`	История и состояние репозитория	History and state of the repository

Частые проблемы

- **E37: No write since last change.** Файл не сохранён. Нажмите `:wq` или `:q!`.
- **E486: Pattern not found.** Искомого текста нет в текущем файле или окне. Часто поиск запускают не в том файле.
- **Чат или команды «ломаются».** Мешает Punto Switcher. Настройки, затем «Программы-исключения», добавить `WindowsTerminal.exe`.
- **Чат не даёт печатать.** Он открывается в обычном режиме. Нажмите `i`.
- **`:q` не закрывает окно при открытых чате или дереве.** Используйте `:qa!`.
- **Parser could not be created for language.** Не установлен парсер. Выполните `:TSInstall language`, подставив название языка.
- **Подсказок кода нет.** Подождите 10–30 секунд и выполните `:checkhealth vim.lsp`.
- **Ответ модели не приходит.** Проверьте, что Ollama запущена (значок в трее, `ollama ps`).
- **Сборка LaTeX не запускается: «could not find the script engine perl».** Не установлен Perl. См. инструкцию `install-markdown-latex`.

Common problems

- **E37: No write since last change.** The file is not saved. Press `:wq` or `:q!`.
- **E486: Pattern not found.** The text is not in the current file or window. Often the search is started in the wrong file.
- **The chat or commands “break”.** Punto Switcher interferes. Settings, then “Exception programs”, add `WindowsTerminal.exe`.
- **The chat does not let you type.** It opens in normal mode. Press `i`.
- **`:q` does not close the window when the chat or the tree is open.** Use `:qa!`.
- **Parser could not be created for language.** The parser is not installed. Run `:TSInstall language`, replacing it with the language name.
- **No code hints.** Wait 10–30 seconds and run `:checkhealth vim.lsp`.
- **No reply from the model.** Check that Ollama is running (the tray icon, `ollama ps`).
- **The LaTeX build does not start: “could not find the script engine perl”.** Perl is not installed. See the guide `install-markdown-latex`.